Project phase 1

setayesh saeidi 40131851

I group objects by concern (core, catalog, order-fulfilment, etc.) so you can see both the micro-level rationale and the macro-level architecture.

---

1. Cross-cutting design choices

Theme	How it appears in the DDL	Why it matters
Surrogate keys	Every table uses `bigserial`(or the PK of another table) as its primary key.	Uniform, never-changing identifiers avoid composite keys and simplify joins, caching, sharding, and URL design. The `big`variant leaves room for millions of rows.
Time-stamping & auditability	`created_at`, `updated_at`, plus domain timestamps such as `shipped_at`.	Lets you build order-tracking, "recently updated" dashboards, and historical reports without extra tables.
Enumerated domains	Custom enum types `order_status`, `payment_status`, `payment_method`.	Enforces valid states at the database layer, allows expressive CHECK constraints, and is friendlier to ORMs than free-text codes.
3rd Normal Form (3NF) base + selective denormalisation	Separate `inventory`, `product_attribute_values`, etc.	Core entities are modelled once; associative tables handle N-to-M. Only data that can change independently is split out, keeping writes small and preventing update anomalies.
Entity-Attribute-Value (EAV)	The `product_attributes` / `product_attribute_values`pair.	Fulfils the "variable attributes" requirement without expanding the products table every time Marketing invents a new spec (e.g., "lumen output").

---

2. Enumerated Types

`order_status`

`Pending → Shipped → Delivered` or `Cancelled`. Captures a *business process*, letting the application enforce legal transitions while the DB guarantees only valid labels are stored.

`payment_status`

Distinguishes *payment* lifecycle (which may differ from the order's): a payment can fail or be refunded even if the order is shipped.

`payment_method`

Gives a finite, integrable list of processors—handy for analytics ("wallet usage %") and fraud rules.

---

3. User & Identity Domain

`users`

- **Purpose:** Master record for every customer.
- **Key columns:**
  - `email` is `UNIQUE NOT NULL` → natural login.
  - `password_hash` (never store plaintext).
  - Premium membership flags + dates (supporting the "upgrade to premium" requirement).
- **Why separate premium dates?** Lets you offer trial periods, renewals, and pro-rate calculations.

`addresses`

- **FK:** `user_id → users.id`. A customer maintains *many* shipping addresses.
- **`is_default` flag:** Enables quick default selection without sub-queries.
- **Design alternative considered:** inline JSON array. Rejected because relational structure enables address-level analytics (heat-map of cities) and per-address order tracking.

---

4. Catalog Domain

`categories`

- **Self-reference `parent_id`:** Implements an *adjacency list* tree (unlimited depth). Works for DigiKala-style nested categories (Electronics > Mobile > Accessories).
- **Nullable parent:** Root categories have `parent_id NULL`.

`products`

- Contains only properties that apply to *all* products. Variable specs break out into EAV.

- `rating_avg` is denormalised for **read-efficiency**; triggers or application code keep it in sync with `reviews`.

`product_categories`

- **PK (`product_id`, `category_id`).** Classic bridge table for many-to-many.
- Having its own table allows multiple categories per product *and* future category-specific metadata (e.g., category ranking).

EAV: `product_attributes` & `product_attribute_values`

Table	Role
`product_attributes`	Dictionary of attribute names (e.g., "color"), expected `data_type` (string, int, JSON), optional `unit`.
`product_attribute_values`	Composite PK (`product_id`, `attribute_id`) stores the actual value as `text`. The app casts/validates based on `data_type`.

Why EAV? — Because an online marketplace must list wildly different items (SSD vs. T-shirt). Adding a physical column per spec would bloat the catalog and require migrations for every new attribute.

`inventory`

- **1-to-1** with `products`; separate so stock updates don't **lock or bloat** the hot `products` row.
- Enables lightweight `UPDATE inventory` statements from warehouse scanners.

---

5. Shopping Experience Domain

`carts`

- Each user can have **one active cart** (`user_id UNIQUE`).
- When checkout succeeds, the app clones cart items into `order_items` and empties the cart.

`cart_items`

- Bridge table (`cart_id`, `product_id`), holds only `quantity`.
- Separating from `orders` keeps abandoned carts lightweight and lets you analyse drop-off without polluting order history.

discounts

- Captures promo code rules.
- `usage_count` + `max_usage` support global limits (campaign budget).
- Per-user checks are implemented in the app or via a separate join table if you later want *strict* DB-level enforcement.

---

6. Order-to-Cash Domain

orders

- Links **who** (`user_id`), **where** (`shipping_address_id`), **how much** (`total_amount`), **promo** (`discount_id`), and **state** (`status` enum).
- Multiple event timestamps let customer service answer "when was it shipped?" without scanning logs.

order_items

- Keeps a **snapshot** of `price_each` at purchase time so later price changes don't corrupt revenue reports.
- PK (`order_id`, `product_id`) because a product shouldn't appear twice in the same order.

payments

- **1-to-1** (`order_id` PK & FK). Many systems allow multiple partial payments; this design assumes a *single* capture. If you need split payments, move `payments.id` to bigserial and keep `order_id` as plain FK.
- `transaction_ref` stores gateway response for reconciliation.

---

7. Social Proof Domain

reviews

- FK to `users` and `products`.
- **Unique index (`user_id`, `product_id`)** ensures one opinion per user per product—fights spam and duplicate ratings.
- Review timestamps enable "most recent" sorting.

---

8. Relationship Map at a Glance

```
users 1—∞ addresses
     1—1 carts 1—∞ cart_items ∞—1 products
     1—∞ orders 1—∞ order_items ∞—1 products
           │
           └──1 payments
products ∞—∞ categories          (via product_categories)
products 1—1 inventory
products ∞—1 reviews (per user)  (via unique constraint)
products ∞—∞ product_attributes  (via attribute_values)
orders   ∞—1 discounts (optional)
```

Cardinalities:

- **1-to-∞**: `users` ➞ `orders`, `addresses`, etc.
- **∞-to-∞**: `products` ↔ `categories`, `products` ↔ `product_attributes`.
- **1-to-1** (logical): `orders` ➞ `payments`, `products` ➞ `inventory`.

Summary

The schema balances **strict relational integrity** (FKs, enums, unique constraints) with **flexibility** (EAV for product specs, self-referencing categories, denormalised averages). Each relation exists to satisfy a specific user story:

- **Shopper journey —** from browsing (`products`, `categories`) → personalisation (`carts`, `addresses`) → purchase (`orders`, `payments`) → feedback (`reviews`).
- **Operational needs —** stock control (`inventory`), promotions (`discounts`), premium memberships (`users` flags).
- **Analytical clarity —** historical prices, explicit statuses, and audit timestamps for every critical event.

Together, these choices create a robust foundation that can scale feature-wise and data-wise while keeping queries predictable and maintainable.